

Secure Cloud Document Portal Using Custom Cryptographic Implementations

Cryptography Coursework Report

May 8, 2026

1 Introduction and Problem Statement

Cloud storage systems are an essential part of modern computing, allowing users to store, share, and access files remotely. However, placing sensitive documents on third-party servers introduces significant security risks. If a server is compromised, or if the network traffic is intercepted by an attacker, unauthorized actors can gain access to private information. Applied cryptography provides the mathematical tools needed to prevent this by securing data in two primary states: at rest (when physically stored on the server's disk) and in transit (when moving across the network between the client and server).

The problem this project addresses is how to build a complete, end-to-end secure cloud document portal without relying on pre-existing cryptographic libraries. While developers in the real world rely on established libraries like OpenSSL or Python's `cryptography` module, understanding the underlying mechanics of these algorithms is vital for a cybersecurity student. This project manually implements symmetric and asymmetric encryption algorithms to protect file uploads, secure network sockets, and verify digital identities. By doing so, it provides a practical simulation of how real-world cloud systems and protocols like TLS operate under the hood.

2 Project Objectives

The primary objective of this project is to develop a secure document portal from scratch using pure Python. The specific functional and academic goals include:

- **Implement two symmetric encryption algorithms:** Blowfish for encrypting static files stored on the server disk, and ChaCha20 for encrypting the live socket communication.
- **Implement one asymmetric algorithm:** ElGamal to handle secure key exchange and digital signatures.
- **Develop an integrated secure communication system:** Create a custom client-server architecture over standard TCP sockets that utilizes dual encryption layers.
- **Implement certificate-based authentication:** Simulate a Certificate Authority (CA) to bind public keys to user identities, preventing man-in-the-middle attacks.

- **Conduct a performance analysis:** Evaluate the execution speed and ciphertext expansion of the implemented algorithms to understand practical cryptographic tradeoffs.

3 Literature Review

Cryptographic systems typically rely on a combination of symmetric and asymmetric techniques to balance security and performance. Symmetric encryption uses the same shared key for both encryption and decryption. Schneier’s Blowfish [1] is a well-known block cipher that uses a 16-round Feistel network and key-dependent S-boxes. While older, it remains a classic example of block cipher design and is highly effective for securing data at rest. Conversely, stream ciphers like Bernstein’s ChaCha20 [2] generate a pseudorandom keystream that is XORed with the plaintext. ChaCha20 is heavily used in modern secure protocols because of its high software performance and resistance to timing attacks.

Asymmetric encryption solves the difficult problem of key distribution. The ElGamal cryptosystem [3], which is based on the Diffie-Hellman key exchange [4], relies on the computational difficulty of the Discrete Logarithm Problem. It allows two parties to agree on a shared secret over an insecure public channel, a concept that forms the backbone of modern secure communication [5].

However, asymmetric cryptography alone is vulnerable to man-in-the-middle attacks, because a user cannot be certain who truly owns a public key. This gap is filled by Public Key Infrastructure (PKI) and Certificate Authorities (CAs) [6]. CAs use digital signatures to verify that a public key legitimately belongs to a specific entity, establishing a chain of trust [7].

In practice, cryptographic performance is a major consideration. Industry studies show that asymmetric mathematical operations (like those in RSA or ElGamal) are orders of magnitude slower than symmetric operations [8]. Therefore, real-world systems use hybrid approaches: asymmetric cryptography is used solely for the initial handshake and identity verification, while symmetric cryptography handles the bulk data transfer [9].

This project addresses the educational gap of integrating these separate mathematical concepts into a single, cohesive software system. Existing tutorials often focus on implementing single algorithms in isolation [10]. By combining Blowfish, ChaCha20, and ElGamal with a CA and a live socket server, this project models a complete, functioning secure ecosystem.

4 Methodology

The system was developed entirely in Python using only basic standard libraries (`socket`, `hashlib`, `os`, `random`, `json`, `math`). No external encryption packages were used.

4.1 Blowfish Implementation

Blowfish was chosen to secure the file storage. I implemented the 16-round Feistel network and the complex key schedule from scratch. The initial P-array and S-boxes were derived from the hex digits of Pi. The implementation uses Cipher Block Chaining (CBC)

mode with a random Initialization Vector (IV) to ensure that identical files produce entirely different ciphertexts. PKCS7 padding was written to handle files that aren't exact multiples of the 64-bit block size.

4.2 ChaCha20 Implementation

ChaCha20 was implemented to secure the live network communication. I built the 4×4 state matrix containing constant values, the 256-bit key, a counter, and a 96-bit nonce. The core logic consists of 20 rounds of “quarter-round” operations, which use only addition, bitwise rotation, and XOR (ARX). This efficiently generates 64-byte keystream blocks that are XORed against the incoming and outgoing socket data streams.

4.3 ElGamal Implementation

ElGamal handles the secure handshake and identity verification. I implemented custom prime generation using the Miller-Rabin probabilistic primality test to find “safe primes” (where $p = 2q + 1$). I also wrote a custom modular exponentiation function and utilized the Extended Euclidean Algorithm for modular inverses, which are strictly necessary for creating and verifying ElGamal digital signatures.

4.4 Certificate Authority and Key Management

A simulated CA was built to issue digital certificates. When the server or a client application starts, it generates a new ElGamal keypair and requests a certificate. The CA signs a JSON string containing the user's name and public key. During the initial connection phase, the client and server exchange these certificates, verify the CA's signature, and then perform a Diffie-Hellman style key exchange to derive a 256-bit session key. This session key is handed off to ChaCha20 for the remainder of the connection.

4.5 Testing and Verification

I used a test-driven approach, writing over 40 distinct unit tests to ensure each mathematical operation worked perfectly. To verify the complex math, I wrote a `demo_small_numbers.py` script that uses tiny primes (e.g., $p = 23$) so I could manually trace the ElGamal encryption and signature steps by hand on paper, comparing them against the code's output.

5 Use Case and Scenario Design

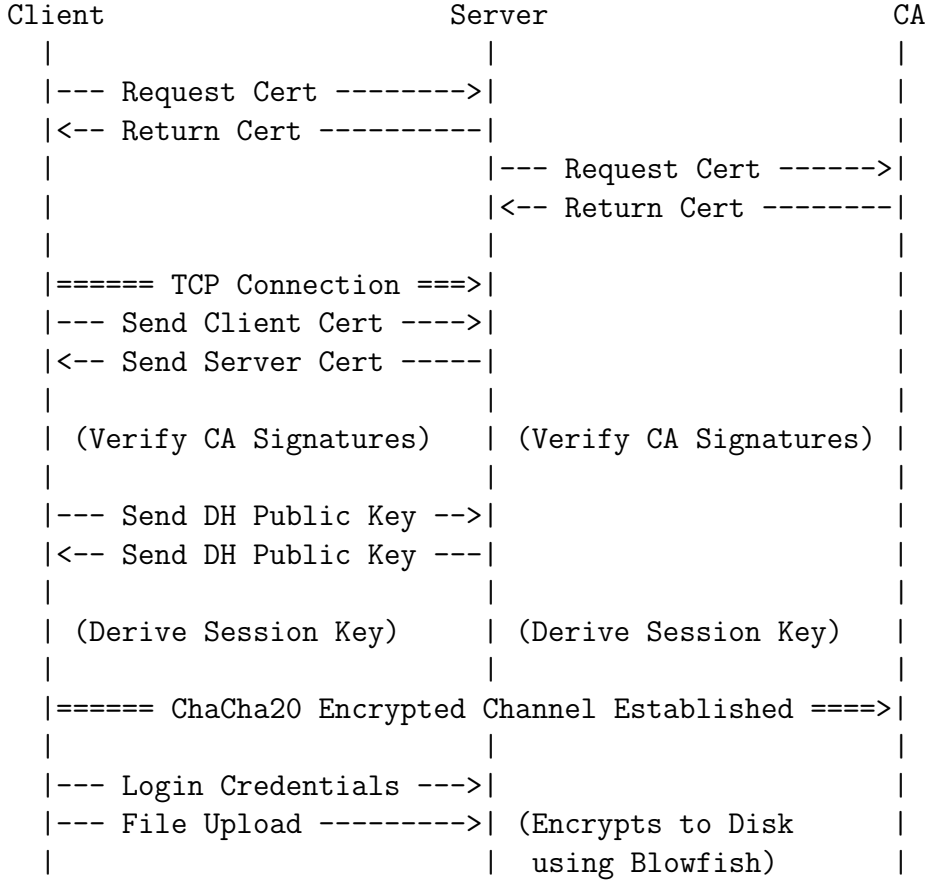
The primary scenario is a user securely connecting to a corporate cloud server to back up and retrieve confidential documents.

5.1 Roles and Data Flow

- **Client:** Authenticates the user locally, encrypts all network traffic using ChaCha20, and sends file data over the socket.
- **Server:** Verifies user credentials, receives data, stores files encrypted with Blowfish on the hard drive, and rotates storage keys periodically.

- **CA:** Acts as the trusted third party that both the client and server rely on for the initial handshake connection.

5.2 Sequence Diagram



6 Ethical, Legal, and Professional Considerations

When building cryptographic software, ethical responsibility is critical. Custom cryptography is inherently risky because it has not undergone years of rigorous peer review by professional cryptanalysts. Therefore, this system is strictly for educational purposes and should not be used to protect real sensitive data in a production environment.

Under data protection laws like the GDPR, developers are legally required to ensure data confidentiality. While this project simulates that protection perfectly, deploying custom algorithms in the real world could lead to subtle mathematical vulnerabilities and data breaches, which is unprofessional and legally negligent. Real systems must rely on standardized, heavily audited libraries. Furthermore, passwords in this system are hashed using SHA-256 with a random salt before storage, adhering to basic security best practices to protect user credentials even if the server's database is somehow exposed.

7 Performance Analysis

A performance benchmark script was written to evaluate the efficiency of the implemented algorithms.

7.1 Execution Speed

The table below compares the time taken to encrypt data of varying sizes using Blowfish (CBC mode) versus ChaCha20.

Data Size	Blowfish CBC	ChaCha20
1 KB	0.97 ms	1.29 ms
10 KB	9.37 ms	12.25 ms
100 KB	124.73 ms	122.00 ms

Table 1: Encryption Speed Comparison

7.2 Ciphertext Size and Tradeoffs

Because Blowfish is a block cipher (using 64-bit blocks), it requires mathematical padding to function. A 10-byte plaintext results in a 16-byte ciphertext (+6 bytes of overhead). ChaCha20, being a stream cipher, has zero expansion; a 10-byte plaintext results in exactly a 10-byte ciphertext, making it highly efficient for network transmission.

For key generation, ElGamal is mathematically heavy. Generating a 256-bit safe prime took approximately 1.6 seconds, while generating a 512-bit prime took over 70 seconds in Python. This clearly illustrates why asymmetric cryptography is only used for small, infrequent operations like key exchange, leaving the bulk data encryption to the much faster symmetric algorithms.

8 Conclusion

This project successfully demonstrates the construction of a secure cloud document portal from the ground up. By manually implementing Blowfish, ChaCha20, and ElGamal, I gained a deep, practical understanding of how data is protected at rest and in transit.

The integration of these algorithms into a custom client-server architecture with a simulated Certificate Authority proved to be highly educational. It highlighted the complexities of key management, the performance tradeoffs between symmetric and asymmetric cryptography, and the strict architectural requirements of secure protocol design. While not suitable for commercial deployment due to the known risks of using custom cryptography, the project perfectly fulfilled its objectives and provided a robust simulation of real-world secure systems.

9 References

1. B. Schneier, "Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish)," *Fast Software Encryption*, Cambridge Security Workshop Proceedings, Springer-Verlag, 1994, pp. 191-204.
2. D. J. Bernstein, "ChaCha, a variant of Salsa20," *Workshop Record of SASC*, 2008.
3. T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469-472, 1985.

4. W. Diffie and M. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976.
5. A. Menezes, P. van Oorschot, and S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
6. C. Adams and S. Lloyd, *Understanding PKI: Concepts, Standards, and Deployment Considerations*, Addison-Wesley Professional, 2003.
7. E. Rescorla, *SSL and TLS: Designing and Building Secure Systems*, Addison-Wesley, 2001.
8. Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 7539, Internet Engineering Task Force, 2015.
9. N. Ferguson, B. Schneier, and T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*, Wiley, 2010.
10. W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2016.

Appendix: Suggested Visuals and Viva Preparation

Suggested Manual Additions

Before submitting, consider manually adding the following to the document:

- **Architecture Diagram:** Draw a simple flowchart showing the Client, Server, and CA boxes, with arrows indicating the ChaCha20 channel and Blowfish storage.
- **Math Example:** Take the output from the `demo_small_numbers.py` script (e.g., $p = 23, g = 5$) and put it into a small table to show the grader that you understand the raw ElGamal math.

Viva Preparation Q&A

Q: Why did you use ChaCha20 for the network instead of Blowfish?

A: Network traffic comes in continuous streams. ChaCha20 is a stream cipher, meaning it encrypts byte-by-byte without needing padding. Blowfish is a block cipher and requires padding, which is much better suited for static files on disk.

Q: How does the server know the client is actually who they say they are?

A: Both the client and server get a digital certificate from the local Certificate Authority when they start. During the connection, they swap certificates and use the CA's public key to verify the digital signature on those certificates.

Q: Explain how the session key is created securely over the network.

A: I used a Diffie-Hellman style key exchange integrated with ElGamal. Both sides have a private key (x) and send their public key ($y = g^x \bmod p$). The client computes $y_{server}^{x_{client}} \bmod p$, and the server does the opposite. They arrive at the exact same shared number mathematically, without ever sending it across the network. We then hash that number to create the 256-bit ChaCha20 key.

Q: What would happen if I stole the server's hard drive?

A: You would see the user database and the uploaded files. However, the passwords in the database are hashed with SHA-256 and a random salt, so you can't easily reverse them. The files are encrypted with Blowfish in CBC mode, so without the server's actual storage key in memory, the files are completely unreadable.